

OmniFlow — Phase 5 (Real-Time Viz) Audit Charter

Research- and council-grounded constraints the Sentinel will grade against. The service is a streaming gateway + a Next.js/R3F front end rendering the live P2P flow for an MD demo and for hiring managers.

Strategy (from the LLM Council)

- **Readable-first, 3D-only-for-topology.** The P2P flow is a node-edge graph — one of the few legitimate 3D uses (spatial/flow, better recall). But 3D *hurts* quantitative reading, so **numbers go on a flat HUD overlay, never painted into 3D space**. Build the flat animated graph first; escalate to a restrained 3D tilt only if it beats the 2D version.
- **Time-box ~1 week core**; record the demo EARLY on seeded data (interviewers watch a recording, not your gateway). Keep a **fallback pre-recorded clip** — live WebGL on a locked-down corporate browser during an MD pitch is fragile.
- **The judgment IS the signal.** Choosing SSE over WebSocket and documenting why reads as more senior than an unjustified bidirectional stack.

Non-negotiable constraints (graded)

1. **Transport = SSE, not WebSocket.** A Go SSE gateway (`franz-go` consumer) subscribes to `omniflow.p2p.completed.v1 / omniflow.inventory.*` and pushes to the browser. Unidirectional server→client → SSE is simpler, auto-reconnects, no ping/pong. Document the SSE-vs-WS decision in an ADR. Only introduce WS if a genuinely bidirectional feature (e.g. server-driven replay control) needs it — and justify it.
2. **Jitter-free timeline.** Client buffers events to a lateness cutoff keyed on the HLC `sequence_engine_key` + the changefeed `resolved` watermark; render only up to the watermark; **order by `sequence_engine_key`, never arrival**; interpolate between states (LERP positions, SLERP rotations) using drei/three helpers — do NOT hand-roll.
3. **Rendering/perf (60fps).** Next.js + React Three Fiber; `InstancedMesh`/drei `<Instances>` for nodes (one draw call); **animated positions in refs, updated in `useFrame` — NOT React state** (the #1 R3F perf killer); `frameLoop='demand'`; **fixed/isometric camera** for the exec view (not free orbit — occlusion/disorientation); LOD/frustum culling.
4. **MD-grade readability.** Plainly-labeled stages (PO Created → Vendor Confirmed → Approved → In Transit → Received); packets flowing along edges as events arrive; status by **color + shape + label** (colorblind-safe, never color alone); a flat HUD with 1–3 live figures (orders processed, \$ committed, SLA breaches). No spinning cameras, glass/chrome, particle screensaver effects.
5. **Real-system fidelity + the two leverage features.** Drive the viz from the ACTUAL event stream, not a synthetic skin. Add (a) **scrub-back / time-travel replay** over the

append-only Kafka/CRDB log — cheap (the log exists) and it visually demonstrates event-sourcing mastery that's otherwise invisible; (b) **business-semantic overlays** (SLA breach, three-way-match failure, cash delta) for the MD. Stretch-only (a second project, don't let it eat the week): an always-on public demo URL + open-sourcing the R3F renderer.

6. **SSE event contract.** Define a lean projection schema (workflow/aggregate id, stage enum, `sequence_engine_key`, `occurred_at`, status, parent/edge, `trace_parent`) derived FROM the existing protobuf events — not a new source of truth.
7. **Deliverables discipline.** ADR (SSE choice + watermark buffering), a couple of front-end/gateway tests, and the fallback clip. Record early. Don't wire the real Kafka pipeline before the render loop is proven.

Failure modes I will hunt

Free-orbit camera; React state driving `useFrame`; WebSocket where SSE suffices (unjustified over-engineering); numbers embedded in 3D (unreadable); color-only status (colorblind-unsafe); live-only demo with no fallback; a synthetic skin disconnected from the real backend; scope explosion (building the public URL / OSS component before the core demo even records).